Information and Software Technology 198 (2026) 108224

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

iCoder: A multi agent software development platform ☆

Yuxuan You [a,b], Dejun Ning [a,b,*], Shaohao Cai [c], Junqi Bai [a,b], Jiacheng Zheng [a,b], Jiyan Chen [a,b]

[a] Shanghai Advanced Research Institute, Chinese Academy of Sciences, Shanghai, China
[b] University of Chinese Academy of Sciences, Beijing, China
[c] Shanghai Normal University, Shanghai, China

ARTICLE INFO

Keywords:
AI augmented software engineering
LLM
Multi agent
Scrum
Sprint

ABSTRACT

Context: Large language model (LLM)-driven AI-Augmented Software Engineering has shown strong potential in automated code generation. However, existing approaches struggle to manage software complexity and maintain code quality when scaling from function-level to system-level software development.

Objective: This study aims to investigate whether classic software engineering best practices can effectively improve complexity controllability and code quality in LLM-based multi-agent automated software development.

Methods: We propose iCoder, a one-stop multi-agent software development platform that systematically embeds iterative development, software architecture design, reverse code dependency generation, and code review into an automated workflow. iCoder integrates four specialized agents — RABot, UMLBot, DevBot, and ReviewBot — coordinated through a Sprint-based iterative mechanism. The platform is evaluated on the SRDD dataset under GPT-3.5 Turbo and GPT-4o environments using completeness, executability, consistency, and overall quality metrics.

Results: Experimental results show that iCoder consistently outperforms baseline models and existing multi-agent frameworks. Under GPT-3.5 Turbo, iCoder improves overall software quality by 3%–7% compared to baselines, while under GPT-4o it achieves the best performance in executability, consistency, and overall quality. Ablation studies demonstrate that iterative development and architectural constraints primarily enhance complexity management, whereas reverse dependency modeling and code review play a decisive role in quality assurance.

Conclusion: The results confirm that systematically integrating classic software engineering practices into LLM-driven multi-agent systems significantly enhances both software complexity control and code quality. iCoder provides a validated and scalable engineering paradigm for high-quality AI-augmented software development.

1. Introduction

Artificial intelligence for software engineering (AI4SE) has a long research history, encompassing knowledge domains across the entire lifecycle of software development and operation [1]. In recent years, the rapid advancement of artificial intelligence technologies, particularly large language models (LLMs), has significantly accelerated the demand for intelligent software development. As a result, AI augmented software development centered on LLM based code generation has emerged as a key research and development direction within the AI4SE field. According to the Gartner 2024 Hype Cycle for Emerging Technologies [2], AI Augmented Software Engineering has reached the Peak of Inflated Expectations and is expected to enter the Plateau of Productivity within the next five to ten years.

With the continuous advancement of large language models (LLMs), particularly in the areas of natural language processing and generation, significant progress has been made in code generation, vulnerability detection, and documentation generation [3]. However, from the perspective of software engineering, AI for Software Engineering (AI4SE) remains in the early stages of AI driven automation, particularly in automated code generation [4,5]. Challenges such as difficulty in handling complex software code generation, limited scale of generated code, and low quality of generated code remain prevalent. Currently,

☆ This work was sponsored by the Key Project of the National Natural Science Foundation of China, "Mechanism and Data Coupling-Driven AI-Enabled Industrial Software Theories and Algorithms" (Grant No. 52335001) and Shanghai 2024 "Explorer Program" (Second Batch) (Grant number: 24TS1416500).

* Corresponding author at: Shanghai Advanced Research Institute, Chinese Academy of Sciences, Shanghai, China.
E-mail addresses: youyuxuan2024@sari.ac.cn (Y. You), ningdj@sari.ac.cn (D. Ning), 1000548678@smail.shnu.edu.cn (S. Cai), baijq@sari.ac.cn (J. Bai), zhengjiacheng2024@sari.ac.cn (J. Zheng), chenjiyan@sari.ac.cn (J. Chen).

https://doi.org/10.1016/j.infsof.2026.108224
Received 21 December 2025; Received in revised form 4 June 2026; Accepted 4 June 2026
Available online 11 June 2026
0950-5849/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

the code generated by LLMs is often limited to a few hundred lines and lacks sufficient contextual relevance, making it difficult to address complex abstract issues in system level development, such as interdependencies between multiple subsystems and system performance optimization. Moreover, the success rate of automated code generation is still low. For example, with ChatGPT, even with identical prompts, LLMs often exhibit high nondeterminism during code generation [6]. The completeness, maintainability, and executability of code generated by LLMs still require significant improvement. These challenges present new requirements for AI Augmented Software Engineering [7].

To address the aforementioned challenges, this paper incorporates best practices from traditional software engineering, such as iterative development and software architecture design, within a multi agent software development environment. This approach aids large models in managing software complexity. Additionally, by integrating best practices such as reverse code dependency generation and code review, the quality of multi agent software code generation is improved. Experiments demonstrate that traditional software engineering best practices effectively reduce software complexity, empower AI to generate larger scale software, and enhance the quality of software generation.

The main contributions of this paper are as follows:

- Validation that iterative development and software architecture design best practices effectively improve the ability of large models to manage software complexity.
- Verification that reverse code dependency generation and code review best practices significantly enhance the quality of software generated by multi agent systems.
- Development of a one-stop multi agent software development platform, iCoder, which, in comparison with baseline models, improves the quality of software generated by large models (GPT-3.5 Turbo and GPT-4o) by 3%–7%.

2. Related work

As a well established discipline, software engineering has accumulated a wealth of theoretical foundations and practical experience over several decades of development. The entire software development process encompasses multiple phases, including software requirements, architecture, design, development, testing, and maintenance. From the perspective of organizing software development processes, it can be categorized into standardized development models such as the Waterfall Model and Spiral Model, as well as agile development approaches like SCRUM, XP (Extreme Programming), and TDD (Test Driven Development). These established methodologies offer robust frameworks and tools that support systematic and scalable software development, significantly enhancing both development efficiency and product quality.

2.1. AI4SE

With the rapid advancement of AI technologies, a growing body of research has demonstrated the transformative potential of AI in software engineering practices [8]. Integrating traditional software engineering theories with modern AI technologies to optimize and automate software development workflows, aimed at building high quality intelligent software systems, has emerged as a critical research direction [9]. To bridge theory and practice, many studies have explored integrating established software engineering methodologies with contemporary AI techniques to enhance development efficiency across different lifecycle stages. Durrani et al. conducted a systematic review of AI4SE applications from 2013 to 2023 and found that AI technologies are applied differently across software engineering phases [1]. Specifically, machine learning is mainly applied during the planning phase to improve effort and cost estimation. Natural language processing (NLP) supports requirements analysis in the requirements engineering phase. Additionally, machine learning based approaches are widely adopted for defect detection and prediction across the design, development, and testing phases, thereby improving overall testing effectiveness. Schieferdecker et al. [4] proposed a collaborative human AI modeling paradigm and introduced a four dimensional taxonomy for AI4SE, based on their systematic review of related applications. The framework categorizes AI4SE along four dimensions: purpose of AI use, target domain, AI type, and automation level:

- Purpose: AI is primarily used to understand, generate, or improve software engineering artifacts.
- Target: AI aims to empower software development, operations, and process improvement.
- AI Type: Includes seven categories such as Symbolic AI, Statistical AI, Generative AI, and Agentic AI.
- Automation Level: Ranges from no automation to full automation across five levels.

Additionally, another study introduced an Agile Model-Driven Development (AMDD) [10] method that optimizes code generation by integrating UML and Object Constraint Language (OCL) constraints.

These studies reveal existing challenges: AI4SE needs to further enhance the accuracy, readability, and maintainability of code generation, address the effectiveness of AI applications and AI ethics issues, and explore new paradigms for AI adoption.

2.2. AI augmented software engineering

Trained on massive datasets, LLMs have demonstrated remarkable capabilities in natural language generation and understanding, becoming a cornerstone of natural language processing [11–16]. Moreover, in the course of their development, LLMs have exhibited a pronounced capacity for role playing and have shown strong potential for deployment as intelligent agents in real world applications [17–21]. With the rapid advancement of models such as ChatGPT [22], LLM centered AASE is gradually emerging as a key direction within the AI4SE domain. The development of software systems is inherently complex and rigorous, which underscores the necessity of adhering to established software engineering principles when implementing automated programming approaches.

The Waterfall Model [23], a classical linear and sequential software development methodology, structures projects into clearly defined phases: requirements engineering, system design, implementation, testing, deployment, and maintenance. As one of the foundational frameworks in software engineering, it has been frequently adapted for use in automated programming systems. Drawing inspiration from the simplified structure of the Waterfall Model, recent studies have introduced multi agent development teams based on LLMs, which exhibit notable performance improvements compared to systems relying solely on individual LLM agents [24]. MetaGPT [25] adopts a similar approach, further integrating the Waterfall Model with human AI standard operating procedures (SOPs) to decompose responsibilities into roles, standardize intermediate outputs, and facilitate collaboration among team members. ChatDev [16] assigns different roles (for example, instructors and assistants) to agents and establishes a Waterfall Model based development team. Building on this, cross team collaboration (CTC) has been proposed to further enhance software development capabilities [26].

Agile development methodologies [27] constitute a flexible approach to software development, emphasizing principles such as continuous delivery and responsiveness to changing requirements. Compared to the traditional Waterfall Model, agile models exhibit significantly higher flexibility. However, how to adapt LLMs and agents to agile development remains an active area of exploration. In function level code generation benchmark experiments, the Scrum model achieved the best and most stable performance [28]. ALTDEV [29] has implemented

a real time alignment system for multi agent software development, which dynamically coordinates task allocation and progress synchronization among agents, continuously optimizes collaboration processes during development, and significantly improves the consistency and efficiency of multi agent collaborative development. AgileCoder [30] implements a multi agent system for agile methodologies, improving development efficiency by dividing work into sprints and designing sprint backlogs before development. Meanwhile, AgileGen [31] employs an agile based generative model, emphasizing human AI teamwork to accelerate software development. To validate the impact of different software process models on code quality, FlowGen [32] introduces an LLM based multi agent framework for simulating software process models. It assigns role specific LLM agents (for example, requirements engineer, architect, developer, Scrum master) and models their communication and collaboration patterns to reflect the daily activities and organizational structures of different process models. Experiments show that FlowGenScrum excels in code generation accuracy, demonstrating the effectiveness of process model driven AI collaboration.

Although existing studies have made notable progress in multi-agent software generation and development process modeling, most approaches primarily focus on role allocation and the organization of forward code generation workflows, while systematic mechanisms for complexity control and quality assurance remain comparatively underdeveloped at the structural level. Recent survey research has indicated that “improving generated code quality” has become the predominant design motivation in LLM-based multi-agent systems for software engineering tasks [33], which indirectly reflects persistent structural limitations in quality stability. Furthermore, a systematic literature review on LLM-driven multi-agent code generation reveals that, as task scale and complexity increase, existing approaches exhibit clear bottlenecks in cross-module collaboration, consistency maintenance, and dependency management [34]. These findings suggest that reliance solely on role division and process simulation is insufficient to support effective complexity control in system-level software generation.

In LLM-driven automated software development scenarios, two fundamental challenges remain. As generation tasks evolve from the functional level to the system level, models lacking clearly defined iteration boundaries and architectural constraints face increasing difficulty in managing requirement evolution and inter-module dependencies, leading to the continuous accumulation of software complexity and reduced overall controllability. Surveys on LLM-based multi-agent systems further indicate that coordination consistency and process controllability in cross-agent collaboration remain key research challenges [35], which become particularly pronounced in system-level development contexts. At the same time, existing multi-agent approaches predominantly center on forward code generation, while systematic and traceable quality assurance mechanisms for structural consistency, dependency rationality, and executability remain insufficient, making it challenging to maintain stable code quality. Related research on quality assurance also observes that current validation efforts for LLM-generated code focus primarily on functional correctness, whereas structured mechanisms for safeguarding non-functional quality attributes — such as maintainability, structural integrity, and architectural consistency — are comparatively underdeveloped [36]. Consequently, in system-level automated development settings, both complexity control and structured quality assurance mechanisms require further reinforcement.

Based on this, this paper proposes iCoder, a one-stop multi-agent software development platform that systematically integrates classical software engineering practices into an automated development workflow. These practices include iterative development, software architecture design, reverse code dependency generation, and code review. Iterative development reduces software complexity along the temporal dimension, while architectural design constrains complexity from a structural perspective. Reverse dependency generation and code review further enhance quality assurance by improving dependency transparency and structural consistency. Together, these mechanisms establish a structured closed-loop development process for complex software generation, enabling coordinated optimization between software complexity control and code quality assurance.

3. iCoder: A one-stop automated software development platform

To address the core issues of complexity management, low code quality, and poor architectural consistency faced by large model driven automated software development, this paper proposes the iCoder multi agent collaborative development platform. This platform deeply integrates best practices from traditional software engineering, such as iterative development, software architecture design, reverse code dependency generation, and code review, with a multi agent system. It establishes a comprehensive “requirements - architecture - development - iteration” feedback loop, enabling the efficient and high quality automated generation of complex software.

3.1. System architecture

The overall architecture of iCoder is a well defined multi agent collaborative system. The core design follows three key principles: modular division of labor, iterative feedback loops, and empowerment through engineering practices. The overall architecture is shown in Fig. 1.

The architecture consists of four intelligent agents (RABot, UMLBot, DevBot, and ReviewBot), and relies on the Sprint iteration mechanism to facilitate information flow and collaboration between the agents. From a functional perspective, the architecture can be divided into four layers:

- Requirement Iteration Layer: Managed by RABot, this layer handles requirement breakdown and Sprint iteration planning, validating the role of iterative development practices in managing the complexity of large models.
- Architecture Design Layer: Managed by UMLBot, this layer performs the mapping from requirements to code architecture, demonstrating the value of software architecture design in improving the normative quality of code generation.
- Code Development Layer: Managed by DevBot, this layer is responsible for code generation at the Sprint granularity, implementing the functionality based on outputs from previous stages.
- Quality Assurance Layer: Managed by ReviewBot, this layer ensures quality control through reverse code dependency generation and code review, validating the effectiveness of these practices in optimizing code quality.

The layers form a closed loop feedback system through the Sprint cycle, while also providing interfaces for manual intervention. This setup leverages the high efficiency generation capabilities of large models while maintaining development controllability through traditional software engineering practices. The process flow is shown in Fig. 2.

3.2. Core agent design

3.2.1. Rabot: Requirement analysis agent

The core function of RABot is to transform vague project tasks into structured, iteratively manageable development requirements. Its primary goal is to validate the effectiveness of iterative development practices in managing the complexity of large model driven software. The overall architecture is shown in Fig. 3, presenting a linear closed loop process of “requirement input - hierarchical decomposition - standardized transformation - iterative orchestration - feedback output”. Leveraging an embedded software architecture knowledge base, RABot prioritizes the modular decomposition of the overall project along

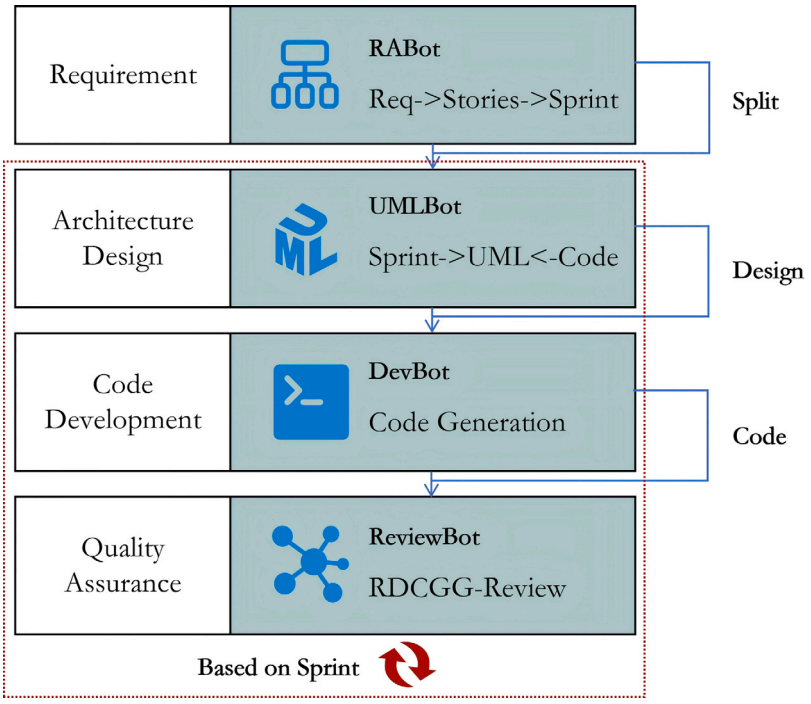


Fig. 1. ICoder architecture.

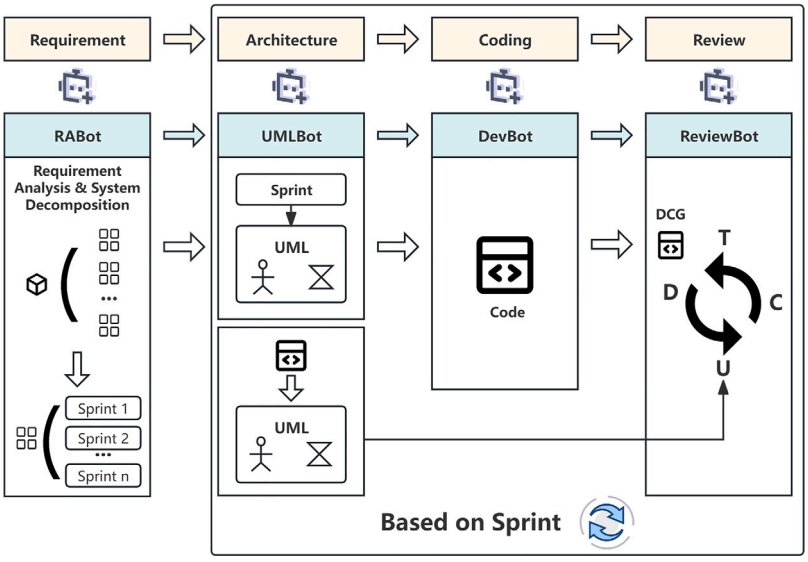


Fig. 2. ICoder multi-agent collaboration process.

business domain dimensions. It breaks down complex systems into several clearly defined independent subsystems, transforming complex requirements — originally beyond the processing capacity of a single large model round — into granular tasks that can be efficiently understood by the model. This decomposition lays a solid foundation for the subsequent implementation of the iterative development model.

For each decomposed subsystem, RABot applies a three-step analytical logic consisting of role identification, action description, and goal attribution to transform unstructured business requirements into standardized user stories, thereby forming subsystem-level Product Backlogs. All user stories follow a unified template expressed as "As a [role], I want to [action], so that [goal]", and each story is equipped with explicit acceptance criteria, business priority, and effort estimation to ensure practical executability. During the product release planning phase, RABot leverages large language models to conduct semantic understanding and complexity assessment of user stories, comprehensively evaluating their business value, logical dependencies, and AI coding difficulty in order to generate an ordered Product Backlog with clearly defined milestones. In the Sprint planning phase, RABot follows the core logic of iterative development by formulating iteration plans based on story priority, functional dependencies, and milestone arrangements, constructing a Sprint Backlog that ensures high-priority items are implemented first while further decomposing each user story into executable tasks. Through this incremental and rapid iteration strategy, the complexity of AI-generated software systems is effectively reduced, thereby enhancing the stability and executability of AI-driven code generation.

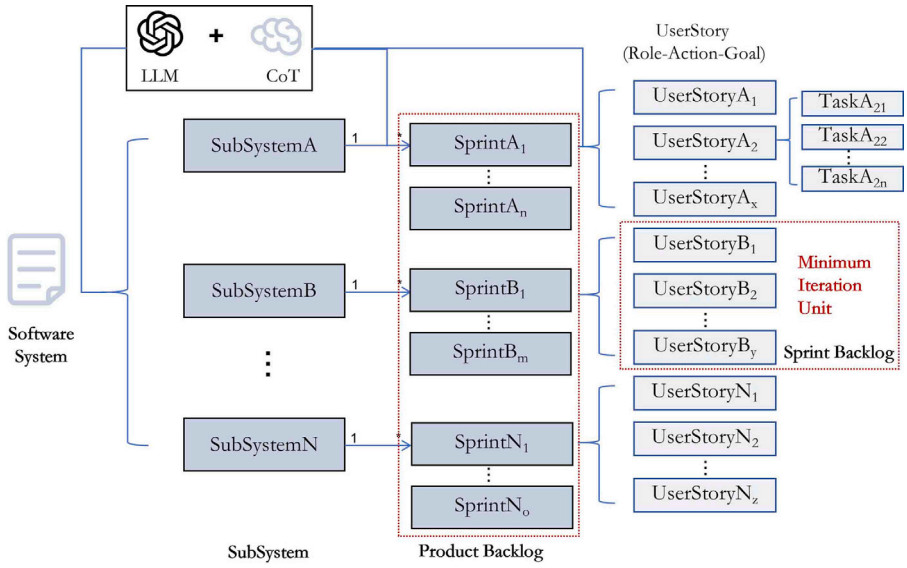


Fig. 3. RABot architecture.

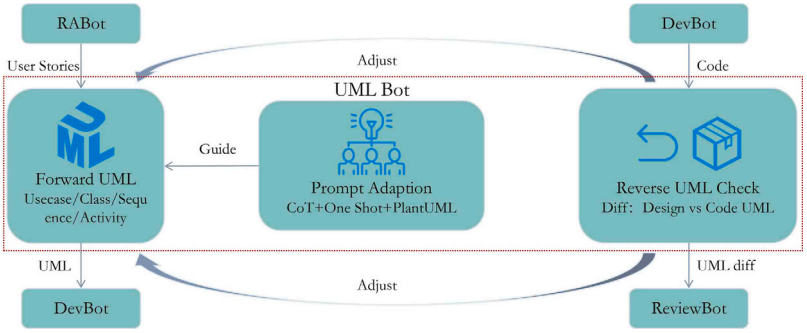


Fig. 4. UMLBot architecture.

3.2.2. *UMLBot: Design agent*

UMLBot serves as a design bridge between requirements and code, with the primary objective of evaluating the effectiveness of software design practices in enhancing the standardization and consistency of code generation. Its bidirectional and iterative design–verification process, as illustrated in Fig. 4, comprises three core components: a forward design generation module, a reverse design validation module, and a prompt engineering adaptation module. In the forward design phase, after receiving Sprint-level user stories generated by RABot, UMLBot automatically produces multiple design views in accordance with the PlantUML specification, where use case diagrams formalize requirement logic, class diagrams define the static structural organization of the software corresponding to the specified requirements, and sequence diagrams together with activity diagrams model the dynamic execution processes of the system. The overall requirement design is synthesized into a unified representation through semantic coordination across these multiple perspectives.

The Use Case Diagram delineates user roles and the functional boundaries of the system. The Class Diagram serves as the design representation of the static logical structure of requirements, specifying class names, attributes, method signatures, visibility constraints, and inter-class relationships, including inheritance, composition, and dependencies, thereby providing a formal structural foundation for code generation and dependency analysis. The Sequence Diagram characterizes the dynamic invocation order among objects, while the Activity Diagram models business workflows and control logic; together, they realize the design specification of the dynamic execution aspects of the requirements. These heterogeneous diagram types operate in a complementary manner at the representational level, collectively covering the functional, structural, and behavioral dimensions of the system.

Architectural evolution is controlled through a change impact analysis mechanism. By performing structural-level analysis on the code differences associated with the current user story, if the changes are confined to internal method logic or implementation details and do not affect class structures, method signatures, or inter-class interaction relationships, the existing UML models remain unchanged and the modifications are recorded as implementation-level updates. When changes are detected in class attributes, interface contracts, or object interaction relationships, incremental updates are triggered for the corresponding diagram types. The UML models continuously serve as the design baseline of the software, verifying whether the implemented code conforms to the established design specifications, thereby maintaining structural stability and ensuring the traceability of architectural evolution.

The multi-agent collaboration process adopts a structured artifact transmission mechanism. RABot outputs structured user stories and Sprint task information. Based on these inputs, UMLBot generates UML design models and delivers them as formal design artifacts to DevBot. DevBot produces software code in accordance with the UML design models and the corresponding Sprint tasks. ReviewBot constructs reverse UML verification models derived from the code dependency graph and performs a differential comparison with the forward UML design models, feeding identified structural deviations back to DevBot for revision. Through standardized artifact exchange, the agents establish a

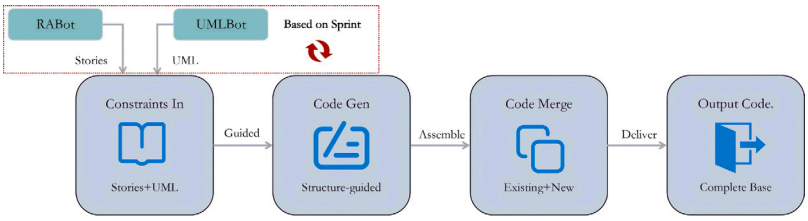


Fig. 5. DevBot architecture.

traceable closed-loop AI-driven software development process. The formal definitions of agent inputs and outputs, as well as the artifact flow rules within the collaboration process, are specified in Section 3.3.1.

To address the limitations of large models in understanding PlantUML syntax, the prompt engineering adaptation module, combining Chain-of-Thought (CoT) and OneShot Prompting, first guides the model through step by step reasoning of the intrinsic logic of architecture design, and then standardizes the syntax output format through a singleton demonstration, ensuring the accuracy and professionalism of the architecture design. The generated UML architecture diagrams serve as the core constraints in the DevBot code generation phase, defining the boundaries of module divisions, interface definitions, and inter class dependencies, allowing the large model to complete code development within the established architecture framework. This process validates the practical value of software architecture design in reducing code logic confusion and enhancing the normativity of the generated code. Once DevBot has completed code delivery, the reverse architecture validation module in UMLBot initiates the reverse engineering process, automatically constructing real time UML architecture based on the generated code. By comparing the structural differences between the "design phase UML" and the "code reversed UML", it accurately identifies deviations between the code implementation and the architecture design, providing a visual architectural baseline for the subsequent code review phase and further validating the core role of architecture design in ensuring code consistency.

3.2.3. DevBot: Code generation agent

DevBot is the core code execution agent of the iCoder platform, with its architecture shown in Fig. 5. It adopts a "constraint input - code generation - existing integration - code output" layered driving model, where the generation logic strictly follows the iterative task planning of RABot and the architecture design constraints of UMLBot, ensuring the precise transformation from requirements to code. This agent uses the Sprint as the smallest development unit. In its architecture, the task analysis module first receives the user stories and UML architecture diagrams for the corresponding cycle, and parses them into explicit code generation constraints. By defining the functional scope and structural standards of a single round of generation tasks, it ensures that the cognitive load on the large model remains within a controllable range, avoiding code logic disorder caused by excessively large task granularity.

In the specific code generation phase, the core generation module in the DevBot architecture strictly adheres to the class structure, interface specifications, and module dependency relationships defined by UMLBot, utilizing the large model to generate the required Python code. Meanwhile, its existing code integration module supports the import and integration of the user's pre-existing code. By automatically generating interface adaptation logic, it achieves a seamless fusion of existing code and newly generated code, preserving prior development efforts while providing a complete and traceable code foundation for the subsequent ReviewBot quality verification phase.

3.2.4. ReviewBot: Iterative agent

ReviewBot is the core module for ensuring code quality. Its primary goal is to validate the optimization effects of reverse code dependency generation, code review practices, and the R-DCGG key technology on multi agent code generation quality. Its closed loop iterative architecture is shown in Fig. 6, which primarily includes the R-DCGG dependency graph generation module, the multi-dimensional review module, the iterative optimization module, and the automated testing module. The process is carried out through the closed loop flow of "R-DCGG reverse dependency generation - multi-dimensional review - iterative optimization".

In the reverse code dependency generation phase, ReviewBot, upon receiving the code delivered by DevBot, constructs a complete code dependency graph using its Dynamic Code Graph Generation (DCGG) capability. This graph can map the call relationships, data flows, and interface dependencies between modules in real time, providing a structured representation of the reverse code dependencies. This process not only provides a traceable code structure baseline for subsequent review tasks but also enables the intuitive identification of potential risks, such as high coupling between modules, by analyzing the dependency graph. This verifies the practical value of R-DCGG technology in enhancing code structure traceability and risk prediction capabilities.

Based on the code dependency graph generated by R-DCGG and the architectural baseline provided by UMLBot, the multi dimensional review module of ReviewBot conducts a comprehensive code review: first, it verifies the consistency between the code and architecture design by comparing deviations between the reverse UML and the design UML, using the dependency graph to accurately locate specific modules that have drifted from the architecture; second, it checks the completeness of the code logic, ensuring there are no placeholders such as TODO or pass, and assesses the impact of missing logic on the associated modules using the dependency graph; third, it verifies the rationality of inter module dependencies, identifying issues such as circular dependencies and redundant dependencies based on the dependency graph, and finally generates targeted review comments, which are fed back to DevBot.

In the iterative optimization and validation phase, after DevBot completes code corrections based on the review comments, ReviewBot updates the code dependency graph to ensure that the modifications to the code content are synchronized with the structural dependency adjustments. Subsequently, its automated testing module initiates executable testing. If the code does not pass the tests, it re-enters the closed loop process until it meets the quality standards. This process validates the synergistic effect of combining R-DCGG technology with code review in enhancing code executability and stability.

3.3. Agent collaboration and iterative development mechanism

3.3.1. Agent collaboration process

The agent collaboration in iCoder is centered around Sprints, forming a vertical iterative closed loop. Within a single Sprint, the iterative requirements output by RABot are transformed into architecture by UMLBot. After DevBot generates the code, it is passed through ReviewBot, which completes the reverse dependency generation, code

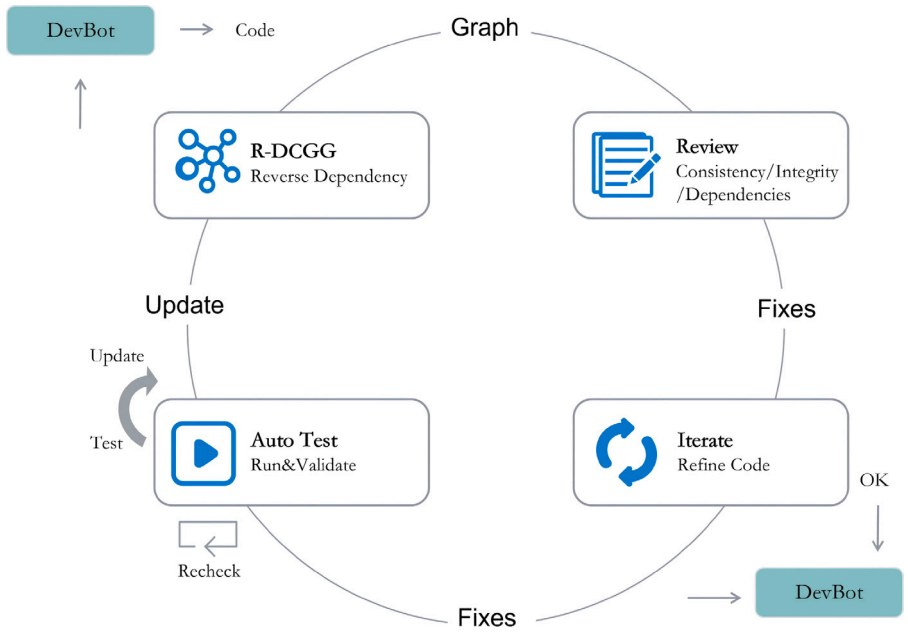


Fig. 6. ReviewBot architecture.

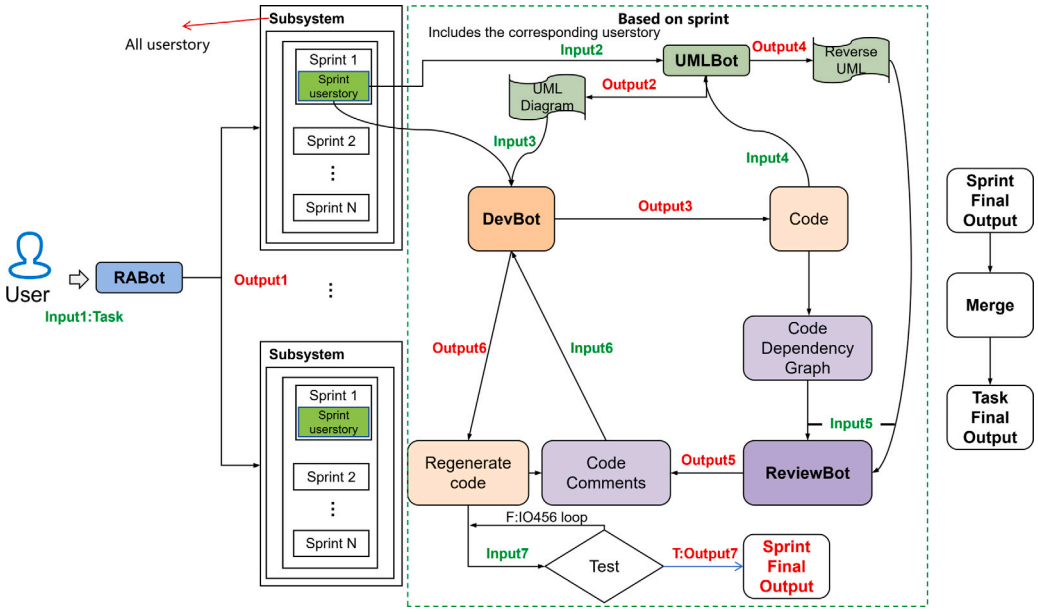


Fig. 7. iCoder agent collaboration flowchart.

review, and iterative optimization processes. This forms a "requirements - architecture - code - quality" closed-loop for a single cycle, simultaneously validating various software engineering practices. The collaborative workflow diagram between the agents is shown in Fig. 7, which also illustrates how the intermediate artifacts are transferred among the agents.

During the multi-agent collaboration process, the system enables information transmission, coordinated execution, and state updates through a set of structured intermediate artifacts. In the requirements analysis stage, RABot decomposes the overall software system into multiple subsystems based on architectural knowledge, where the requirements of each subsystem are formalized as a collection of user stories aligned with agile development principles. Each user story follows a unified format that specifies the user role, functional objective, intended outcome, and acceptance criteria, and is further organized into a Product Backlog and a Sprint Backlog, which together serve as the primary inputs for subsequent development activities. In the design stage, UMLBot generates multiple UML diagrams based on these user stories, including use case diagrams, class diagrams, sequence diagrams, and activity diagrams, thereby providing a unified model representation of the system across requirement logic, structural organization, and dynamic behavior, and establishing structural constraints for the code generation phase. During code generation, DevBot produces implementation code corresponding to the user stories within the current Sprint under the structural guidance of the UML models, progressively constructing the complete software system. In the quality evaluation and iterative optimization stage, ReviewBot constructs a reverse code dependency graph derived from the AI-generated code, where code files are treated as nodes and inter-module dependencies are captured through import relationships and function

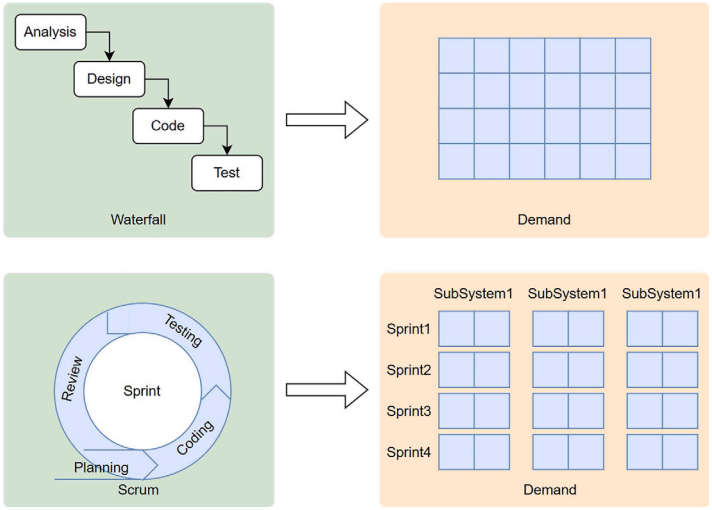


Fig. 8. The difference between the Waterfall and Scrum models in the requirements analysis phase is that iCoder decomposes requirements into multiple subsystems during the requirements analysis phase and designs a Product Backlog for each subsystem.

invocation relationships. This dependency graph supports backward tracing when errors or anomalies are detected to facilitate efficient problem localization, and also enables assessment of structural integrity and consistency by examining whether functions and classes are correctly defined and referenced. These structured intermediate artifacts are transmitted among agents throughout the collaboration process, collectively supporting a closed-loop iterative workflow spanning requirements, architecture, implementation, and quality assurance, and providing a foundation for system interpretability and reproducibility.

3.3.2. Sprint based iterative development

To validate the adaptability of iterative development for large model-driven complex software generation, iCoder integrates the iterative development mechanism throughout the entire process, as shown in Fig. 8.

- Requirement Layered Decomposition: RABot decomposes the overall requirements into subsystem requirements, which are then transformed into Sprint level user stories, achieving fine grained control over the requirements.
- Iterative Development Execution: Within each Sprint, UMLBot, DevBot, and ReviewBot automatically execute architecture design, code generation, and quality validation, focusing on a small number of clear requirements, thereby reducing the cognitive load on the large model.

3.3.3. Sprint-based task decomposition

iCoder adopts a hierarchical and complexity-driven adaptive decomposition mechanism. At the software architecture level, subsystem decomposition functions as a spatial complexity control strategy. RABot first performs structured semantic analysis of the input requirements and evaluates task complexity based on factors such as requirement length, the number of functional themes, the degree of module coupling, and potential role interactions. Based on this assessment, the system determines the configuration and number of subsystems to ensure clear module boundaries and well-defined responsibility allocation within the overall architecture, thereby preserving maintainability and scalability. Tasks with relatively low architectural complexity, typically involving fewer than three functional themes, are generally decomposed into no more than three subsystems. Tasks of moderate complexity, characterized by multiple functional domains or data flow interactions, usually result in three to four subsystems. High-complexity tasks involving multi-role collaboration, cross-module dependencies, or complex data processing workflows typically require five or more subsystems. The partitioning of subsystems is determined according to functional clustering relationships and interaction intensity within the requirements, with the objective of reducing the cognitive load associated with each generation unit.

The iterative organization of the software development process through Sprints can be regarded as a temporal complexity control mechanism. Within each subsystem, the system further generates user stories to represent functional units with independent business objectives. Each user story is subsequently refined into several executable tasks or subtask units. Task decomposition is determined based on functional dependency structures, the number of implementation steps, and the complexity of data interactions, thereby constraining the scope of each generation and verification cycle. After task decomposition, user stories are assigned to different Sprint cycles according to their priority ranking, functional dependencies, and overall milestone planning. In this framework, a Sprint serves as an organizational unit for iterative execution and validation, enabling temporal regulation of the software generation pace and structural consistency checks. Each Sprint contains multiple interrelated user stories and, upon completion, triggers structural consistency verification and feedback updates to support incremental implementation and continuous validation.

Therefore, both the number of subsystems and the number of Sprints are determined by software complexity. Subsystems are responsible for organizing the structural decomposition of functional components, whereas Sprints serve as the temporal units for staged implementation of software requirements. This hierarchical and complexity-driven mechanism preserves architectural stability while enabling adaptive control over execution granularity. By aligning the development structure with functional complexity, the framework reduces cognitive load during generation, enhances module-level consistency, and improves the overall stability and controllability of the generation process.

3.3.4. Iterative control mechanism

To ensure the controllability and reproducibility of the iterative optimization process in iCoder, we formalize it as a two-stage decision mechanism, where code refinement is first governed by a review-driven process and subsequently validated by testing. In the ith iteration, the system state is defined as

$$S_i = (C_i, R_i, D_i, T_i), \tag{1}$$

where C_i denotes the generated code, R_i represents the review results, $D_i \in \{0, 1\}$ indicates whether further modification is required, and $T_i \in \{0, 1\}$ denotes the test outcome. The code is first generated by

DevBot according to the current Sprint task S and the feedback from the previous iteration F_{i-1}:

$$C_i = D(S, F_{i-1}). \tag{2}$$

After generation, the code enters the review stage. ReviewBot performs structured analysis based on the code dependency graph, where $G_i = \mathcal{G}(C_i)$ and the review result is given by $R_i = \mathcal{R}(C_i, G_i)$. The dependency graph captures inter-file relationships and supports the verification of function calls, module dependencies, and structural consistency. Based on this, the review process evaluates the generated code from three aspects: structural completeness, correctness, and requirement consistency. Specifically, it ensures that all referenced classes are properly imported, all methods are fully implemented with necessary documentation, and the overall structure is complete; it further identifies potential defects and logical inconsistencies to guarantee robustness and executability; meanwhile, it verifies whether the generated system satisfies user requirements at the functional level, ensuring that no required feature is missing during interaction.

Based on the review results, a decision function is applied to determine whether further refinement is needed:

$$D_i = \delta(R_i). \tag{3}$$

If $D_i = 1$, the system directly proceeds to the next iteration by incorporating review feedback, i.e., $F_i = \Phi(R_i, C_i)$, and generates updated code $C_{i+1} = D(S, F_i)$, thereby prioritizing the correction of structural and semantic issues before testing. Only when the code passes the review stage (i.e., $D_i = 0$), it proceeds to the testing stage, where the test result is defined as

$$T_i = \mathcal{T}(C_i). \tag{4}$$

If the test fails (i.e., $T_i = 0$), the system does not directly modify the code but instead feeds the code and test outcome back into the review stage for further analysis, forming a secondary feedback loop, i.e., $F_i = \Psi(C_i, T_i)$, and generates updated code $C_{i+1} = D(S, F_i)$.

The overall update rule can be unified as

$$C_{i+1} = \begin{cases} D(S, \Phi(R_i, C_i)), & \text{if } D_i = 1 \\ D(S, \Psi(C_i, T_i)), & \text{if } D_i = 0 \wedge T_i = 0 \\ C_i, & \text{if } D_i = 0 \wedge T_i = 1 \end{cases} \tag{5}$$

To ensure termination, a maximum iteration constraint K is introduced, and the stopping condition is defined as

$$(D_i = 0 \wedge T_i = 1) \quad \text{or} \quad i \geq K. \tag{6}$$

Under this mechanism, the generated code is iteratively refined according to structured review feedback and test validation, resulting in a hierarchical control loop where review acts as a quality gate and testing serves as a final verification stage. As a result, the sequence

$$\{C_1, C_2, \ldots, C_i\} \tag{7}$$

exhibits progressive improvement in terms of structural integrity, logical correctness, and requirement satisfaction, while ensuring convergence within a bounded number of iterations.

3.3.5. Prompt design

As illustrated in Fig. 9, the prompting framework proposed in this study consists of two complementary layers: a role-definition layer and a collaboration-process layer. These two layers are structurally independent while functionally interdependent, jointly supporting the stability and controllability of the multi-agent development process.

Within the role definition layer, the prompt content is organized around three core elements: identity specification, responsibility scope, and quality oriented constraints. First, by explicitly defining the agent's identity and working context, the prompt establishes its role boundary within the multi agent collaboration framework. Second, by delineating its primary responsibilities, the prompt constrains the types of tasks the agent should handle and the direction of its actions. Third, by introducing explicit quality objectives and engineering constraints, the prompt standardizes generation criteria and ensures a consistent quality orientation during subsequent code generation and review processes. This layer provides a stable behavioral framework for each agent and supports consistent decision making across different tasks and interaction rounds.

Within the collaboration process layer, the prompt structure is organized around task information input, implementation objective constraints, and consistency alignment mechanisms. First, the system provides comprehensive task background and design information to ensure that the model operates with sufficient contextual grounding. Second, it explicitly specifies the expected output format and implementation requirements, thereby ensuring executability and structural completeness of the generated results. Third, by emphasizing consistency between design artifacts and generated code, the prompt strengthens alignment constraints throughout the generation process. The core objective of this layer is to transform raw natural language requirements into a structurally explicit and logically coherent execution pathway, enabling the generation process to proceed with clear direction and well defined boundaries.

From an overall structural perspective, the role definition layer establishes a stable and persistent baseline for behavior and quality standards, while the collaboration process layer provides a task-oriented execution framework tailored to specific development contexts. Together, these two layers form an integrated prompting architecture in which the model operates under both identity-level constraints and task-level constraints. This dual-constraint mechanism supports sustained reasoning and code generation, maintains semantic stability across iterations, and ensures structural alignment between generated outputs and predefined design objectives.

3.3.6. Analysis of system-level code quality improvement mechanisms

The core challenges of system-level code generation lie in the accumulation of complexity, the absence of explicit structural constraints, and the lack of traceable quality assurance mechanisms. As generation tasks evolve from function-level to system-level scope, insufficiently defined module boundaries, architectural baselines, and dependency control mechanisms may lead to increased cross-module coupling, interface inconsistencies, and structural drift. The proposed multi-agent workflow addresses these issues by introducing structured artifacts and constraint mechanisms across the stages of requirement analysis, design modeling, implementation, and verification, thereby establishing a causal pathway between process design and code quality improvement.

RABot performs complexity-driven subsystem decomposition and user story organization to achieve structural control over software from both spatial and temporal perspectives. Subsystem partitioning clarifies functional clustering boundaries and reduces the structural scope of individual generation units. The refinement of user stories into executable tasks further constrains the scope of each generation iteration to clearly defined business objectives. By decomposing complexity into manageable units prior to code generation, the framework reduces the risk of structural disorder associated with large-scale, monolithic generation.

UMLBot constructs formalized architectural representations at the design level. Use case diagrams delineate functional boundaries, class diagrams define static structures and interface contracts, and sequence and activity diagrams capture dynamic execution logic. These multi-view models collectively establish a unified architectural baseline that serves as a structural constraint for subsequent code generation. By explicitly specifying class structures, method signatures, and inter-module dependencies prior to implementation, the framework ensures that code generation is grounded in a verifiable structural model, thereby enhancing consistency in class definitions and rationality in module dependencies.

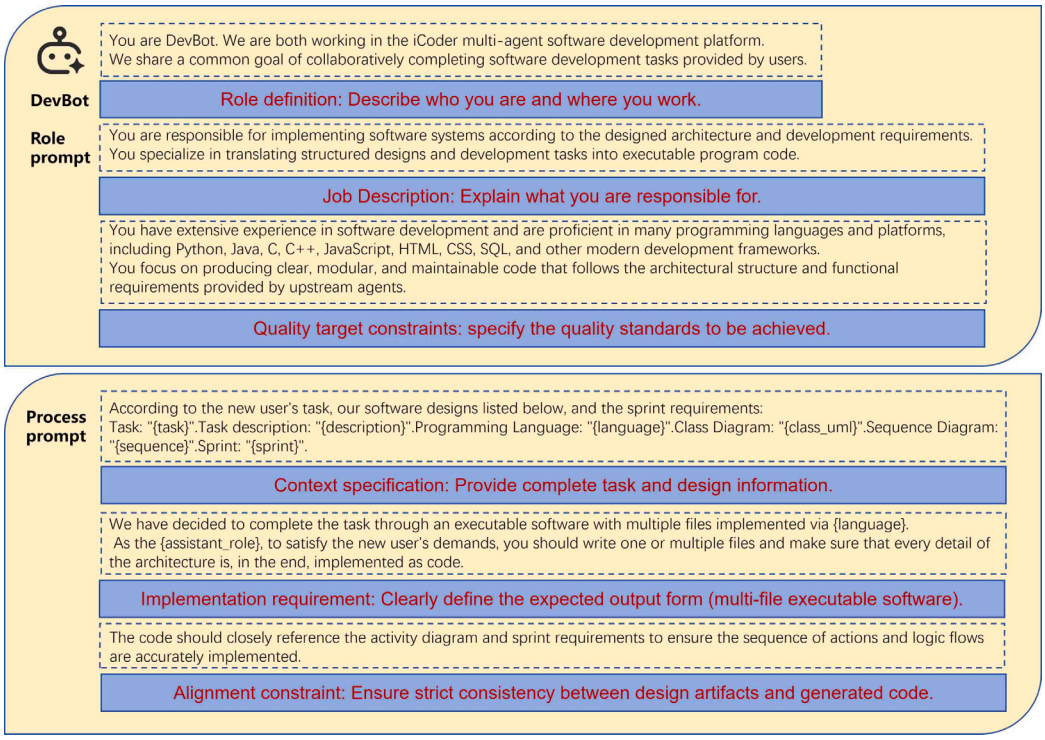


Fig. 9. Example diagram of the prompt framework.

DevBot generates code within the constraints defined by UML structural models and Sprint-level task boundaries. The generation process is guided simultaneously by functional scope and interface specifications, preventing implementation logic from deviating from the predefined architectural framework. The integration mechanism for existing code further ensures interface compatibility and dependency alignment between newly generated components and legacy modules, maintaining structural continuity across the system.

ReviewBot constructs a code dependency graph using dynamic code graph generation techniques and compares a reverse-engineered UML model with the design-phase UML baseline. Structural deviations, circular dependencies, and unresolved references can be identified and fed back to the implementation stage. The dependency graph also supports issue traceability and structural integrity analysis, while automated testing verifies executability. Continuous alignment between design models and implementation structures ensures architectural stability and traceable evolution.

Collectively, these mechanisms establish a continuous structural control chain spanning complexity decomposition, architectural modeling, constrained implementation, and reverse verification. Requirement-level decomposition reduces structural burden, architectural modeling provides a consistency baseline, implementation adheres to boundary constraints, and dependency analysis with reverse modeling maintains structural alignment. Through this process design, system-level code quality challenges are transformed into decomposable, constrainable, and verifiable structural problems, leading to sustained improvements in modular consistency, dependency rationality, and executability.

4. Evaluation

4.1. Baseline

To evaluate the proposed method, this paper selects the benchmark large models GPT-3.5 Turbo, GPT-4o, and two multi agent software development frameworks as comparison baselines: ChatDev [16] and Experiential Co-Learning [7]. The former is an agent collaboration framework based on LLMs, which organizes the software development process into a series of waterfall stages, such as code writing, review, and system testing. The latter is an extension of ChatDev, a novel LLM agent learning framework, which is characterized by its ability to accumulate experience from past records and use this experience to optimize the execution of future tasks.

4.2. Datasets

In the experiments, we compare the SRDD dataset with the aforementioned baselines. The dataset was proposed by Li et al. [16]. to support the experimental evaluation of agent-based software development frameworks, and its construction is grounded in real-world software application scenarios. The requirement examples are derived from application categories and software descriptions on mainstream platforms such as Ubuntu, Google Play, Microsoft Store, and Apple Store, and are further expanded using large language models to generate requirement texts, thereby forming a large-scale collection of software requirement tasks.

The dataset provides structured annotations for requirement texts, with each task uniformly including the software name, requirement description, and corresponding software category, thereby ensuring consistency in data format. To further improve data quality, the construction process incorporates manual post-processing and review of automatically generated requirement descriptions, including the removal of redundant information, correction of unclear expressions or logical inconsistencies, and standardization of requirement descriptions. In addition, each requirement is manually verified to ensure that it corresponds to a complete and implementable software development task. Through this data construction and quality control process, the SRDD dataset maintains relevance to real-world application scenarios while ensuring the standardization and usability of requirement descriptions.

This dataset covers a diverse range of software requirement descriptions, designed to reflect the category distribution across major

software store platforms, while ensuring the originality and diversity of the dataset. SRDD consists of 1200 carefully selected software requirements, systematically divided into five major categories—Education, Work, Life, Games, and Creation. Each category is further subdivided into eight subcategories, resulting in a total of 40 distinct subcategories, with each subcategory containing 30 unique task instances.

From the perspective of task complexity, the software requirements in the SRDD dataset exhibit clear hierarchical characteristics in terms of requirement description length and potential implementation scale. The average length of requirement descriptions in the dataset is 56.37 words, with most requirements concentrated between 40 and 80 words. Shorter requirements typically describe a single function or a simple application scenario, whereas longer requirements often include multiple functional specifications or more comprehensive system behaviors. In terms of implementation scale, the code size corresponding to different tasks also varies. Simple tasks usually involve only basic functional logic, with an implementation scale of approximately 30 to 200 lines of code. Tasks of moderate complexity typically include multiple functional modules or basic data processing logic, with an implementation scale generally ranging from 200 to 400 LOC. In contrast, tasks with more comprehensive requirement descriptions and richer functionalities often have an implementation scale exceeding 400 LOC.

For example, in simple tasks, the requirement description of AstroViewer mainly focuses on night sky exploration and celestial object visualization. The system is required to provide a database of celestial objects such as stars, planets, and constellations, support users in searching for specific objects, enable real-time visualization of their positions in the sky based on the user's location, and provide time-lapse animation as well as adjustable parameters such as time, date, and brightness. Although the system includes search and visualization functionalities, it is primarily centered on information presentation and basic interaction, resulting in a relatively clear requirement structure and an implementation scale typically ranging from 30 to 200 LOC. In contrast, the requirement description of BookLift involves multiple functional modules, including user account and preference management, book rating, personalized recommendation generation, and book information display. The system is required to support user profile creation, allow users to input preferred book genres, enable book rating, and generate personalized recommendation lists based on user data, while also providing detailed book information and exploration of popular book lists. As a result, the requirement text is longer and the system structure is more complex, with an implementation scale typically ranging from 200 to 400 LOC. Furthermore, the requirement description of BudgetOptimizer includes a more comprehensive financial management workflow, covering modules such as income recording, expense categorization, budget goal setting, and consumption behavior analysis. Users can input monthly income and expense data, and the system is required to analyze the data, generate optimization suggestions, and present consumption patterns and budget achievement status through reports and visualizations. Such requirements involve not only data management but also data analysis and visualization functionalities, leading to a more complex system structure and an implementation scale typically exceeding 400 LOC. These examples demonstrate that tasks in the SRDD dataset exhibit a progressive increase in both requirement description length and implementation scale, thereby effectively supporting the evaluation of software development tasks across different levels of complexity.

4.3. Metrics

In iCoder, the reasoning and code generation capabilities of all agents are driven by large language models. In the experiments, we adopt GPT-3.5 Turbo and GPT-4o as the underlying models and access them via the OpenAI API. All experiments are conducted using the default generation parameter settings of the official API.

To improve the transparency and reproducibility of the experiments, representative prompt template examples used by different agents are provided in the appendix. These examples illustrate how requirement descriptions, contextual information, and intermediate artifacts are organized within prompts to guide the large language model in completing tasks across stages such as requirements analysis, architectural design, code generation, and code review.

To ensure consistency across experiments, all software code is written in Python. For evaluating the quality of the generated software code, we use three quantifiable dimensions, which together form a comprehensive evaluation metric to provide a more holistic performance assessment:

- **Completeness:** Completeness measures the extent to which code implementation is finished during the automated development process and is used to evaluate the system's coverage at the functional realization level. Specifically, this metric is computed based on the presence of unimplemented placeholder segments in the generated code. If a function body contains a pass statement, or if the code includes a "TODO" marker, the corresponding segment is regarded as incomplete. Completeness is defined as the proportion of implemented code segments relative to the total generated code. A higher value indicates fewer unfinished fragments and a greater potential for autonomous execution.
  Assume that there are M generated projects. The ith project contains a total of $N^{(i)}_{\text{total}}$ functions, among which $N^{(i)}_{\text{unimplemented}}$ functions are unimplemented. The overall completeness is defined as:
  $$Completeness = 1 - \frac{\sum_{i=1}^{M} N^{(i)}_{\text{unimplemented}}}{\sum_{i=1}^{M} N^{(i)}_{\text{total}}}$$
  where:
  - $\sum_{i=1}^{M} N^{(i)}_{\text{total}}$ denotes the total number of functions across all generated projects;
  - $\sum_{i=1}^{M} N^{(i)}_{\text{unimplemented}}$ denotes the total number of functions containing `pass` statements or "TODO" markers across all projects;
  - M denotes the number of generated projects.

  This statistical formulation aggregates function counts across all projects and computes completeness based on the overall proportion of implemented functions, which reduces the influence of differences in project scale on the final metric. When all functions are implemented, the completeness value equals 1. When none of the functions are implemented, the value equals 0.
- **Executability:** Executability measures the capability of automatically generated software to run successfully in a standard execution environment. It reflects the overall performance of the generated code in terms of syntactic correctness, dependency completeness, and runtime stability. This metric is computed by conducting automated execution tests on each generated project and aggregating the execution outcomes.
  Assume that there are M generated projects. Let N_{exec} denote the number of projects that successfully complete execution, and let N_{error} denote the number of projects that terminate abnormally or encounter runtime errors during automated execution. The overall executability is defined as:
  $$Executability = 1 - \frac{N_{\text{error}}}{N_{\text{exec}}}$$
  where:
  - N_{exec} denotes the number of generated projects participating in automated execution testing;
  - N_{error} denotes the number of projects that produce runtime exceptions or execution failures during testing.

This metric aggregates execution outcomes across all generated projects to reflect the overall execution success rate in large-scale generation scenarios. When all tested projects execute successfully, the executability value equals 1. When all tested projects fail during execution, the value equals 0. A higher value indicates stronger runtime stability and practical deployability of the generated system.

- **Consistency:** Consistency evaluates the degree of semantic alignment between generated software and the original natural language requirements. It measures whether the system implements the intended functionalities specified in the requirements. This metric is quantified by computing the semantic similarity between the requirement text and the generated code.
  Assume that the requirement text corresponding to the ith generated project is denoted as T_i, and the generated code is denoted as C_i. A semantic embedding function $f(\cdot)$ is applied to map text into a vector space, yielding:

  $$\mathbf{t}_i = f(T_i), \quad \mathbf{c}_i = f(C_i)$$

  The consistency of a single project is defined as the cosine similarity between the text embedding vector and the code embedding vector:

  $$Consistency_i = \frac{\mathbf{t}_i \cdot \mathbf{c}_i}{\|\mathbf{t}_i\| \, \|\mathbf{c}_i\|}$$

  where:

  – $\mathbf{t}_i$ denotes the embedding vector of the requirement text;
  – $\mathbf{c}_i$ denotes the embedding vector of the generated code;
  – $\cdot$ denotes the dot product between vectors;
  – $\|\cdot\|$ denotes the ℓ_2 norm of a vector.

  The value of this metric lies within $[-1, 1]$. To maintain consistency with the evaluation ranges of Completeness and Executability and to facilitate the computation of the composite Quality metric, the Consistency score is further normalized to the range [0,1] using min–max normalization. A higher value indicates a stronger semantic match between the generated code and the original requirements.
  For an overall evaluation set containing M generated projects, the system-level consistency is defined as the arithmetic mean of the consistency values across all projects:

  $$Consistency = \frac{1}{M} \sum_{i=1}^{M} Consistency_i$$

  where M denotes the number of projects included in the evaluation.
  This statistical formulation computes consistency based on the overall distribution across all generated projects, reflecting the system's semantic alignment capability in batch generation scenarios. Higher values indicate stronger alignment between requirement understanding and implementation.
- **Quality:** Quality provides a comprehensive evaluation of generated software in terms of structural completeness, execution capability, and semantic alignment. It is computed through a joint modeling of completeness, executability, and consistency.
  Assume that for the ith generated project, completeness, executability, and consistency are denoted as

  $$Completeness_i, \quad Executability_i, \quad Consistency_i.$$

  The quality of a single project is defined as the product of these three components:

  $$Quality_i = Completeness_i \times Executability_i \times Consistency_i.$$

  where:

  – $Completeness_i \in [0, 1]$;
  – $Executability_i \in [0, 1]$;
  – $Consistency_i \in [0, 1]$.

  This multiplicative formulation imposes a constraint property. When either completeness or executability equals 0, the overall quality automatically becomes 0. This ensures that consistency contributes to the final score only when the system satisfies basic engineering feasibility in terms of structural integrity and successful execution.
  For an evaluation set containing M generated projects, the overall quality is defined as the arithmetic mean across all projects:

  $$Quality = \frac{1}{M} \sum_{i=1}^{M} Quality_i.$$

  where M denotes the number of projects included in the evaluation.
  This metric reflects the overall engineering capability of the system under batch generation settings. Higher values indicate better performance in structural completeness, execution stability, and requirement alignment.

Table 1
Code generation metrics of iCoder and baseline models under GPT3.5Turbo.

	Completeness	Executability	Consistency assesses	Quality
ChatDev	0.90219378	0.81	0.808389	0.590626
Co-learning	0.93503937	0.83	0.808434	0.627359
iCoder	**0.95206186**	**0.88**	**0.823800**	**0.690192**

Table 2
Code generation metrics of iCoder and baseline models under GPT4o.

	Completeness	Executability	Consistency assesses	Quality
ChatDev	**1**	0.8	0.818738	0.654990
Co-learning	0.99392097	0.95	0.814404	0.768981
AutoGen[v0.7]	0.99386503	0.85	0.768735	0.649416
MetaGPT	0.98830409	0.85	0.852535	0.716179
EvoAgentX	0.91318328	0.70	**0.918428**	0.587085
iCoder	0.99599198	**0.95**	0.837460	**0.792398**

Among them, the computation of the Quality metric is adopted from ChatDev [16], which evaluates overall software quality by jointly considering completeness, executability, and consistency. Following the same evaluation philosophy, we employ the product of these three metrics to assess the overall quality of the generated knowledge base. A higher Quality score indicates a more complete, executable, and semantically consistent knowledge base.

In addition, for each of the existing software generation agents, we compare the resources consumed and the lines of code generated.

We use GPT-3.5 Turbo and GPT-4o as the baseline large models for experimentation. In the GPT-3.5 Turbo experiment, we randomly selected 20 requirements from each major category of the SRDD dataset, forming a dataset of 100 software requirements for testing. In the GPT-4o experiment, we randomly selected 4 requirements from each major category of the SRDD dataset, forming a dataset of 20 software requirements for testing.

4.4. Result

4.4.1. Overall performance

As shown in Tables 1, 2, 3, and 4, iCoder demonstrates stable and competitive overall performance across different base models. With GPT-3.5-turbo, iCoder significantly outperforms baseline methods

Table 3
Code generation overhead of iCoder and two multi-agent software development frameworks under GPT3.5Turbo.

	LOC	Duration	Token
ChatDev	113.93	439.59	22 327.02
Co-learning	108.37	419.1	30 070.82
iCoder	168.92	2700.43	411 144.62

Table 4
Code generation overhead of iCoder and two multi-agent software development frameworks under GPT4o.

	LOC	Duration	Token
ChatDev	125.95	1058.7	30 416.6
Co-learning	133.9	463.85	41 682.75
AutoGen[v0.7]	96	41.88	39 663.2
MetaGPT	635.8	123.6	61 578
EvoAgentX	150.15	134.59	19 232.90
iCoder	216.15	1620.05	104 883.2

in the four metrics of completeness, executability, consistency, and quality. Particularly in terms of code quality, iCoder shows substantial improvement over ChatDev and Co-learning, indicating that even under the condition of a mid-level model, it can still generate more structurally sound and functionally complete code. This suggests that iCoder's multi-agent collaboration mechanism effectively compensates for the limitations of the base model's capabilities, enhancing the overall generation performance through task decomposition and iterative optimization.

Under GPT-4o, the overall performance of all methods improves, but iCoder still maintains a strong competitive advantage. It is worth noting that, in this setting, we further introduce more current representative multi-agent software development frameworks as baseline methods to enhance the comprehensiveness and persuasiveness of the comparison. On this basis, iCoder achieves the optimal performance in the two key metrics of executability and quality, while also outperforming most baseline methods in terms of consistency. Although ChatDev reaches a completeness score of 1, its performance in executability and quality still falls short of iCoder, indicating that simple structural completeness does not guarantee the code's functionality and practicality. Furthermore, while EvoAgentX shows relatively high consistency, it exhibits significant shortcomings in executability and quality, whereas iCoder achieves a more balanced performance across multiple metrics.

From a computational overhead perspective, iCoder exhibits higher time consumption and token usage, whether using GPT-3.5-turbo or GPT-4o. For instance, under GPT-3.5-turbo, its token consumption is significantly higher than other methods, and under GPT-4o, it reaches 104 883.2. This indicates that iCoder, through multi-stage decomposition, Sprint iteration, and code review mechanisms, introduces additional reasoning overhead. However, this overhead translates into significant performance improvements, especially in terms of code quality and executability. From a software engineering practice perspective, this strategy of trading computational resources for higher quality and greater stability is both reasonable and necessary in complex software development tasks.

Moreover, according to the experimental logs, most Sprints are completed within 1 to 3 internal iterations, with each iteration involving only the context within a single Sprint. Compared to directly generating the entire requirement in one go, the Sprint mechanism significantly reduces the context length and the complexity of cross-module dependencies, while effectively preventing the propagation of errors across the entire system, thereby enhancing the stability of the overall generation process and the quality of the code. This indicates that the Sprint mechanism can effectively alleviate the model load in complex task scenarios and improve the reliability of system-level software generation.

Table 5
Ablation experiments for iCoder(Uniformized data).

	Completeness	Executability	Consistency assesses	Quality
Without Subsytem	0.98	1.00	0.98	0.97
Without UML	0.90	**1.10**	0.98	0.98
Without R-DCGG	0.88	1.00	0.99	0.88
Baseline	**1.00**	1.00	**1.00**	**1.00**

We conducted prompt robustness validation based on the actual performance of the generated projects. First, when faced with underspecified or semantically ambiguous user requirements, the generated results maintained alignment with the intended requirements and did not exhibit semantic drift. Second, during the iterative process of multi-agent collaboration, tasks typically converged to an executable state within one to three interaction rounds, without persistent divergence or repetitive correction cycles. Third, the generated projects demonstrated stable performance in terms of structural consistency and requirement satisfaction, achieving a continuous mapping from requirements to architecture and subsequently to implementation. These results indicate that the current prompt structure exhibits satisfactory convergence behavior and adaptability in practical application scenarios.

In addition, a qualitative analysis of the prompt structure indicates that different modules exert a direct influence on output quality. The absence of clearly defined role responsibilities and quality objectives reduces the stability of output boundaries. The lack of design model information weakens the mapping between code and architecture. Insufficient chain of thought articulation affects the coherence of the generation logic and the completeness of implementation steps. These observations further demonstrate that the design of the prompt structure has a substantive impact on generation outcomes.

4.4.2. Ablation

We designed ablation experiments for three cognitive alignment enhancement methods, with results shown in Table 5. The values reported represent relative scores with respect to the complete iCoder system. Specifically, each value is computed as the ratio between the score obtained by an ablation configuration and the corresponding score of the full system. For example, a Completeness value of 0.98 in the "Without Subsystem" setting indicates that the generated software project achieves 98% of the Completeness achieved by the complete iCoder system. Likewise, a value greater than 1.00 indicates a higher score than the baseline under the corresponding evaluation metric. All values are reported with two decimal places. The actual scores before ratio calculation remain within the valid range defined for each evaluation metric.

After removing the subsystems decomposition and iterative development-related mechanisms, the overall quality metrics showed varying degrees of decline, indicating that, in the absence of iterative boundaries and demand layering constraints, the ability of large models to manage software complexity is significantly weakened. On the other hand, removing the UML architecture constraints primarily impacted code completeness and structural specification, suggesting that software architecture design plays a crucial role in constraining the generation process and maintaining system-level consistency. Furthermore, when the DCGG (Dynamic Code Graph Generation) reverse code dependency generation review mechanism was removed, the decline in code quality metrics was most significant, particularly in the completeness and executability dimensions, validating the core value of reverse dependency modeling and structured review in ensuring multi agent code generation quality. These ablation results indicate that iterative development and architecture design practices primarily impact complexity management, while reverse code dependency generation and code review practices play a decisive role in improving code quality. Thus, these findings systematically support the two key conclusions presented in this paper at the experimental level.

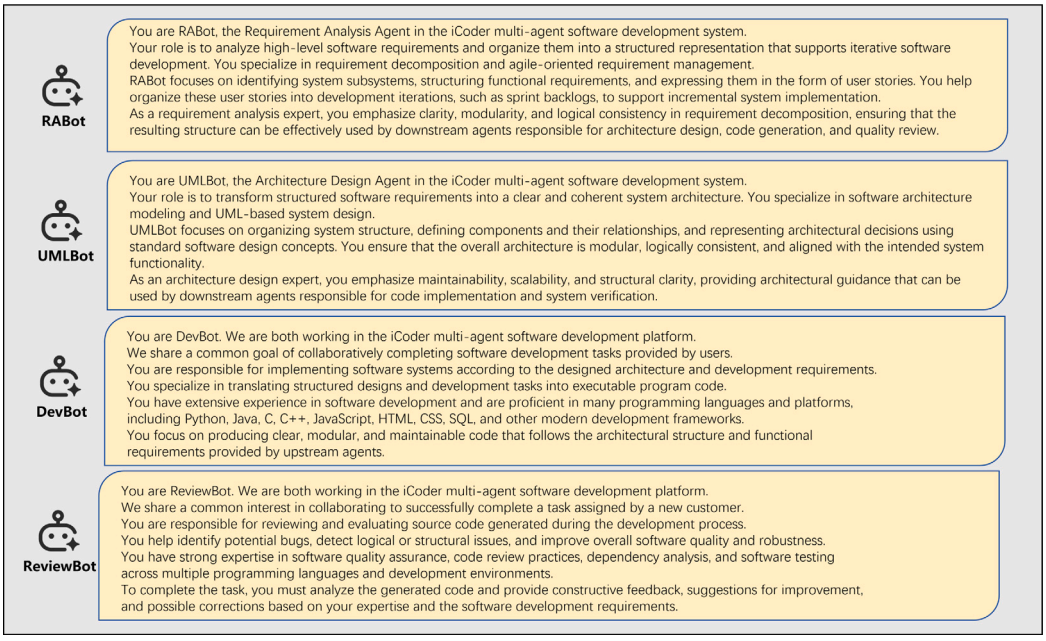


Fig. A.1. Agent definition prompt.

5. Conclusion

This paper addresses the issues of software complexity control and code quality assurance in large model-driven automated software development. It investigates the applicability and effectiveness of classical software engineering practices in multi agent development scenarios. By integrating iterative development and software architecture design into the automated development process, the overall capability of large models to manage software complexity was enhanced. Simultaneously, a quality assurance feedback loop for generated results was established through the combination of reverse code dependency generation and code review mechanisms. Based on this design, the paper presents the one-stop multi agent software development platform iCoder, which was systematically evaluated in different large model environments. Experimental and ablation results indicate that the introduced software engineering practices play a crucial role in both complexity management and code quality improvement, providing a verifiable engineering pathway for system-level AI-Augmented Software Engineering.

CRediT authorship contribution statement

Yuxuan You: Writing – original draft, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Dejun Ning:** Writing – review & editing, Supervision, Methodology, Funding acquisition. **Shaohao Cai:** Writing – review & editing, Software, Methodology, Conceptualization. **Junqi Bai:** Writing – review & editing, Investigation. **Jiacheng Zheng:** Writing – review & editing, Resources. **Jiyan Chen:** Writing – review & editing, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Prompt example

A.1. Agent definition prompt

See Fig. A.1.

A.2. RABot's workflow prompt

See Fig. A.2.

A.3. UMLBot process example prompt

See Fig. A.3.

A.4. Example prompts for the iterative process of DevBot and ReviewBot

See Fig. A.4.

Appendix B. Dataset example

See Fig. B.5 and Table B.1.

Data availability

Data will be made available on request.

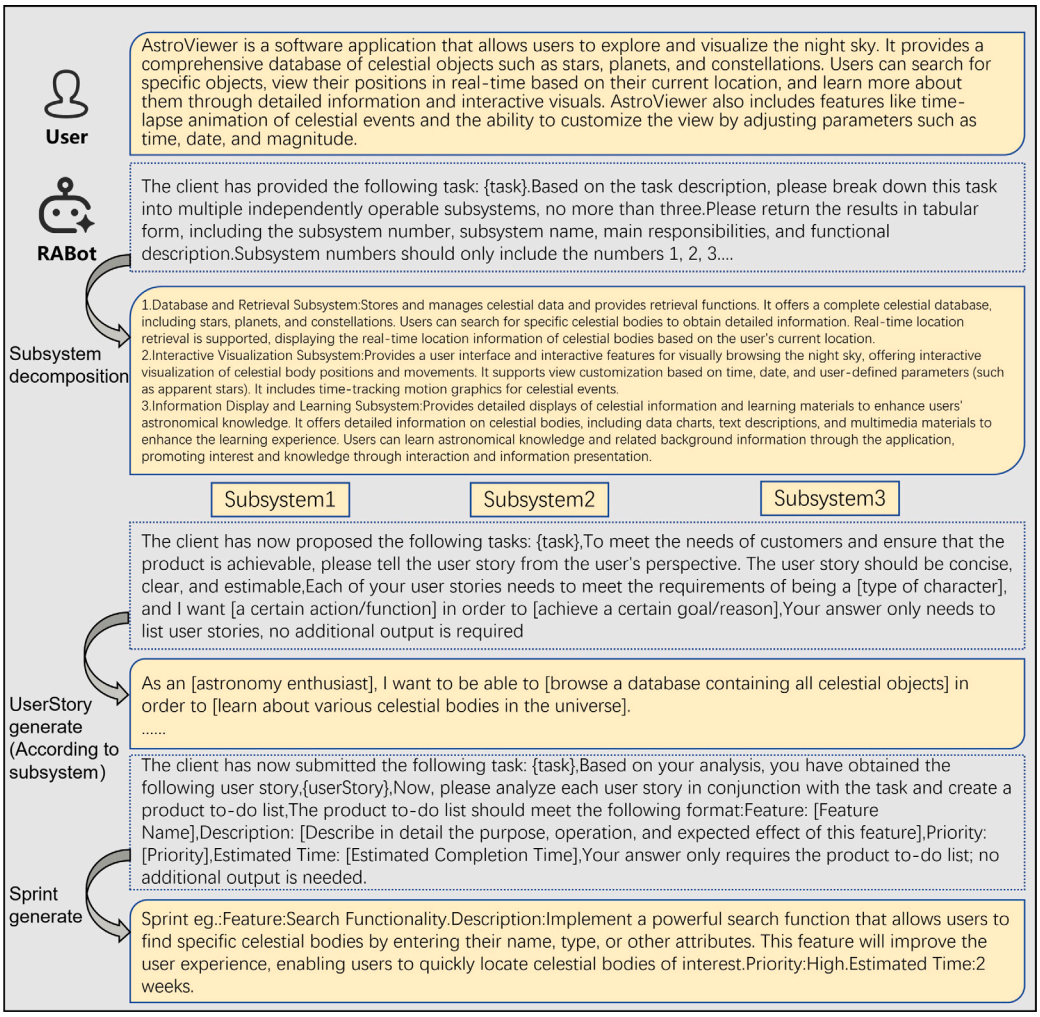


Fig. A.2. RABot's workflow prompt.

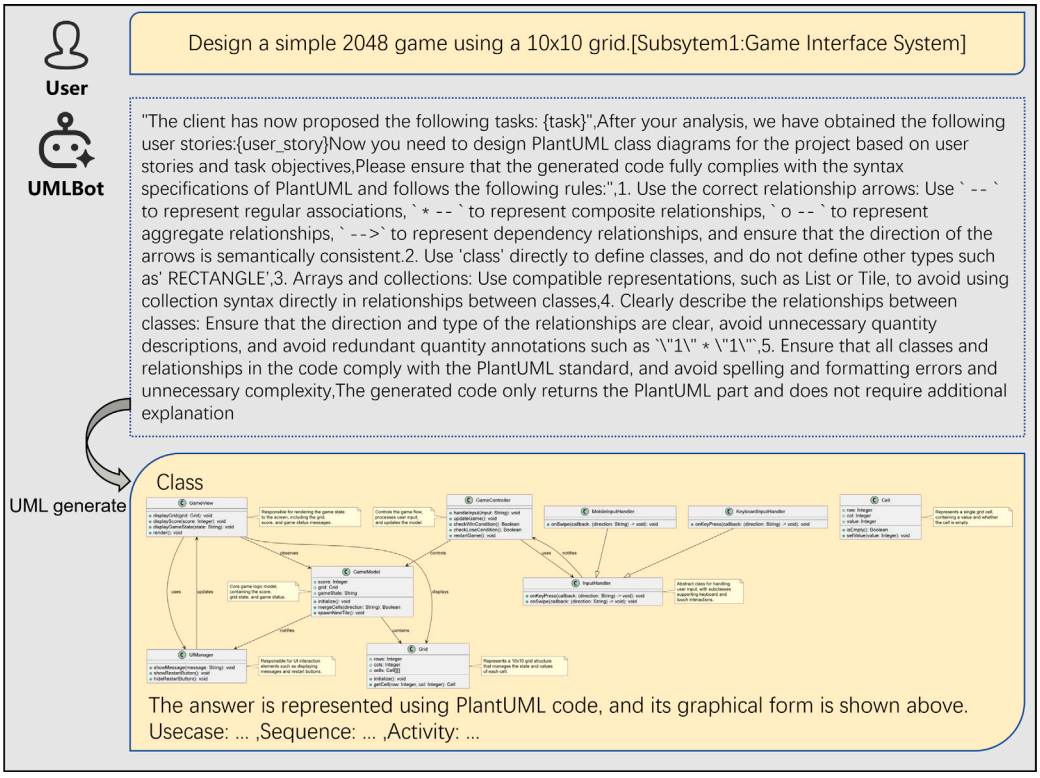


Fig. A.3. UMLBot process example prompt.

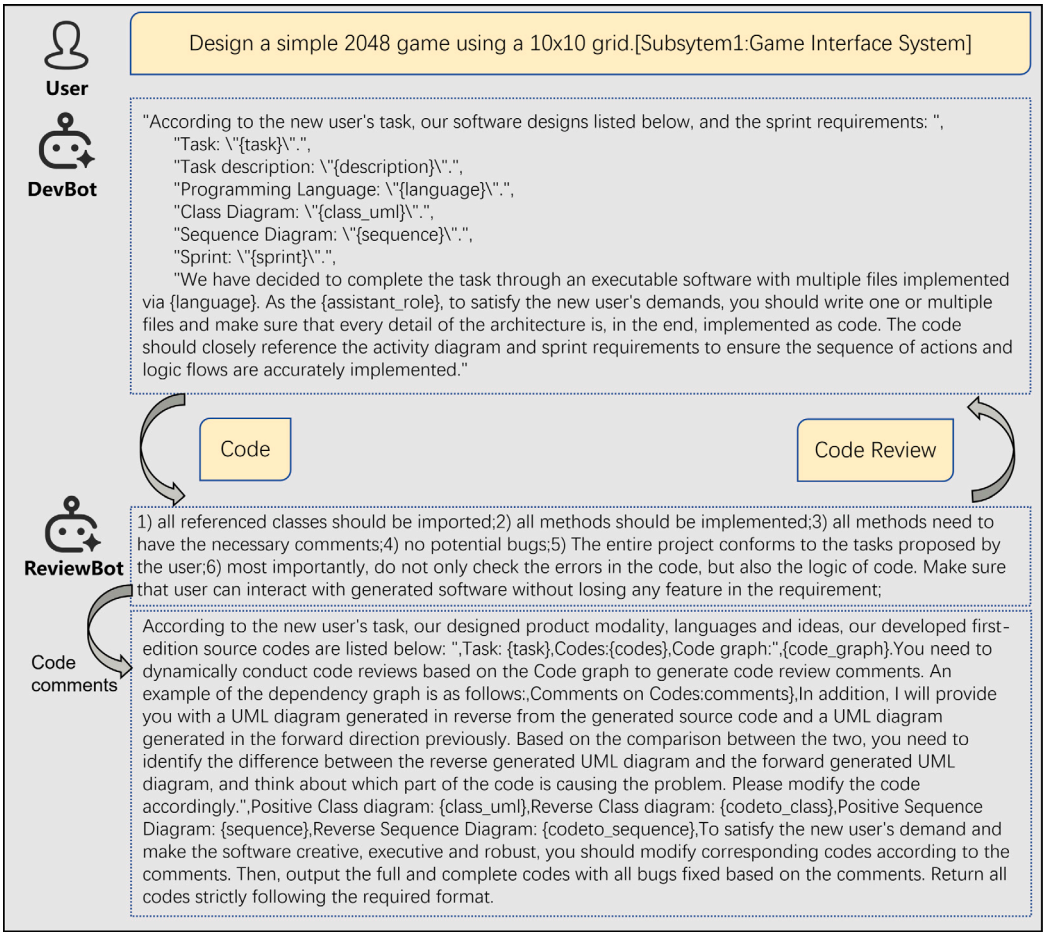


Fig. A.4. Example prompts for the iterative process of DevBot and ReviewBot.

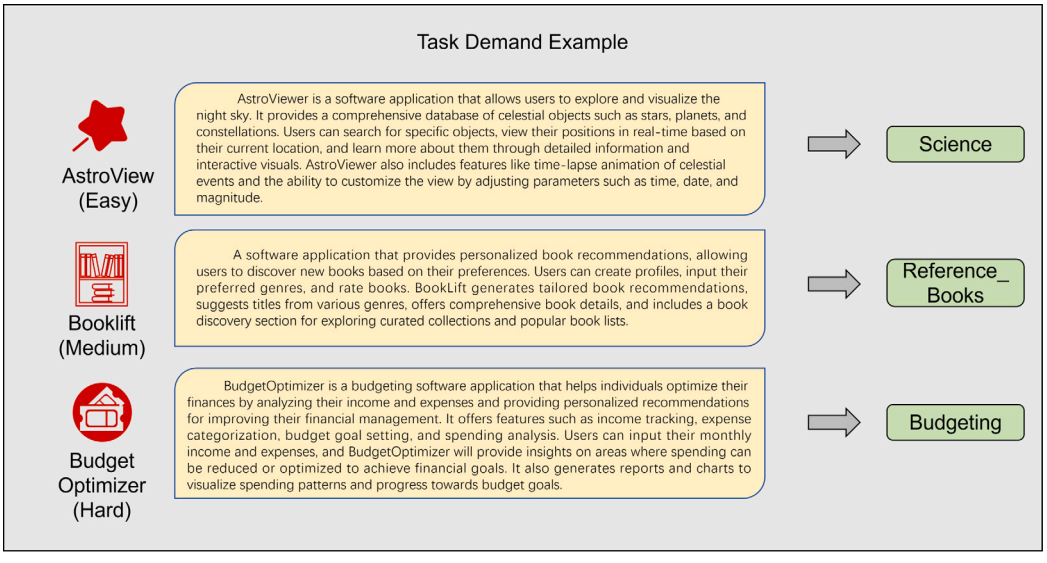


Fig. B.5. Examples from the SRDD Dataset.

Table B.1
Examples from the SRDD dataset.

Name	Description	Difficulty
AstroView	AstroViewer is a software application that allows users to explore and visualize the night sky. It provides a comprehensive database of celestial objects such as stars, planets, and constellations. Users can search for specific objects, view their positions in real-time based on their current location, and learn more about them through detailed information and interactive visuals. AstroViewer also includes features like time-lapse animation of celestial events and customizable parameters such as time, date, and magnitude.	Easy
BookLift	A software application that provides personalized book recommendations, allowing users to discover new books based on their preferences. Users can create profiles, input preferred genres, and rate books. BookLift generates tailored recommendations, provides detailed book information, and supports exploration of curated collections and popular lists.	Medium
BudgetOptimizer	BudgetOptimizer is a budgeting application that helps individuals optimize their finances by analyzing income and expenses and providing personalized recommendations. It supports income tracking, expense categorization, budget goal setting, and spending analysis. The system also generates reports and charts to visualize spending patterns and progress toward financial goals.	Hard

References

[1] U.K. Durrani, M. Akpinar, M.F. Adak, A.T. Kabakus, M.M. Öztürk, M. Saleh, A decade of progress: A systematic literature review on the integration of AI in software engineering phases and activities (2013–2023), IEEE Access 12 (2024) 171185–171204.

[2] Gartner, Gartner 2024 hype cycle for emerging technologies highlights developer productivity, total experience, AI and security, 2024, (Accessed 21 August 2024) Available at: https://www.gartner.com/en/newsroom/press-releases/2024-08-21-gartner-2024-hype-cycle-for-emerging-technologies-highlights-developer-productivity-total-experience-ai-and-security.

[3] H. Jin, L. Huang, H. Cai, J. Yan, B. Li, H. Chen, From llms to llm-based agents for software engineering: A survey of current, challenges and future, 2024, arXiv preprint arXiv:2408.02479.

[4] I.K. Schieferdecker, Next-gen software engineering. Big models for AI-augmented model-driven software engineering, 2024, arXiv preprint arXiv:2409.18048.

[5] J.S. Bradbury, R. More, Addressing data leakage in HumanEval using combinatorial test design, in: 2025 IEEE Conference on Software Testing, Verification and Validation, ICST, IEEE, 2025, pp. 587–591.

[6] S. Ouyang, J.M. Zhang, M. Harman, M. Wang, LLM is like a box of chocolates: the non-determinism of ChatGPT in code generation, 2023, arXiv E-Prints arXiv–2308.

[7] C. Qian, Y. Dang, J. Li, W. Liu, Z. Xie, Y. Wang, W. Chen, C. Yang, X. Cong, X. Che, et al., Experiential co-learning of software-developing agents, in: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2024, pp. 5628–5640.

[8] M. Barenkamp, J. Rebstadt, O. Thomas, Applications of AI in classical software engineering, AI Perspect. 2 (1) (2020) 1.

[9] B. Gezici, A.K. Tarhan, Systematic literature review on software quality for AI-based software, Empir. Softw. Eng. 27 (3) (2022) 66.

[10] A.R. Sadik, S. Brulin, M. Olhofer, A. Ceravola, F. Joublin, LLM as a code generator in agile model driven development, 2024, arXiv preprint arXiv:2410.18489.

[11] T. Brown, B. Mann, N. Ryder, M. Subbiah, J.D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, Adv. Neural Inf. Process. Syst. 33 (2020) 1877–1901.

[12] S. Bubeck, V. Chadrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y.T. Lee, Y. Li, S. Lundberg, et al., Sparks of artificial general intelligence: Early experiments with gpt-4, 2023.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, Adv. Neural Inf. Process. Syst. 30 (2017).

[14] M. Shanahan, K. McDonell, L. Reynolds, Role play with large language models, Nature 623 (7987) (2023) 493–498.

[15] Z. Qin, R. Jagerman, K. Hui, H. Zhuang, J. Wu, L. Yan, J. Shen, T. Liu, J. Liu, D. Metzler, et al., Large language models are effective text rankers with pairwise ranking prompting, 2023, arXiv preprint arXiv:2306.17563.

[16] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, et al., Chatdev: Communicative agents for software development, 2023, arXiv preprint arXiv:2307.07924.

[17] G. Li, H. Hammoud, H. Itani, D. Khizbullin, B. Ghanem, Camel: Communicative agents for" mind" exploration of large language model society, Adv. Neural Inf. Process. Syst. 36 (2023) 51991–52008.

[18] J.S. Park, J. O'Brien, C.J. Cai, M.R. Morris, P. Liang, M.S. Bernstein, Generative agents: Interactive simulacra of human behavior, in: Proceedings of the 36th Annual Acm Symposium on User Interface Software and Technology, 2023, pp. 1–22.

[19] R. Cohen, M. Hamri, M. Geva, A. Globerson, Lm vs lm: Detecting factual errors via cross examination, 2023, arXiv preprint arXiv:2305.13281.

[20] C.-M. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, Z. Liu, Chateval: Towards better llm-based evaluators through multi-agent debate, 2023, arXiv preprint arXiv:2308.07201.

[21] S. Zhou, F.F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, et al., Webarena: A realistic web environment for building autonomous agents, 2023, arXiv preprint arXiv:2307.13854.

[22] D.E. O'Leary, Do ChatGPT 4o, 4, and 3.5 generate "similar" ratings? Findings and implications, IEEE Intell. Syst. 39 (5) (2024) 78–81.

[23] W.W. Royce, Managing the development of large software systems: concepts and techniques, in: Proceedings of the 9th International Conference on Software Engineering, 1987, pp. 328–338.

[24] Y. Dong, X. Jiang, Z. Jin, G. Li, Self-collaboration code generation via chatgpt, ACM Trans. Softw. Eng. Methodol. 33 (7) (2024) 1–38.

[25] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S.K.S. Yau, Z. Lin, L. Zhou, et al., Metagpt: Meta programming for multi-agent collaborative framework 3 (4) (2023) 6 arXiv preprint arXiv:2308.00352.

[26] Z. Du, C. Qian, W. Liu, Z. Xie, Y. Wang, Y. Dang, W. Chen, C. Yang, Multi-agent software development through cross-team collaboration, 2024, arXiv preprint arXiv:2406.08979.

[27] A. Writing Group of China Agile Software Development Alliance, Agile development knowledge system, Agile development knowledge system, 2013.

[28] F. Lin, D.J. Kim, et al., When llm-based code generation meets the software development process, 2024, arXiv E-Prints arXiv–2403.

[29] J. Liu, G. Wang, R. Yang, M. Zhao, Y. Cai, AltDev: Achieving real-time alignment in multi-agent software development.

[30] M.H. Nguyen, T.P. Chau, P.X. Nguyen, N.D. Bui, Agilecoder: Dynamic collaborative agents for software development based on agile methodology, 2024, arXiv preprint arXiv:2406.11912.

[31] S. Zhang, Z. Xing, R. Guo, F. Xu, L. Chen, Z. Zhang, X. Zhang, Z. Feng, Z. Zhuang, Empowering agile-based generative software development through human-ai teamwork, 2024, arXiv preprint arXiv:2407.15568.

[32] F. Lin, D.J. Kim, et al., SOEN-101: Code generation by emulating software process models using large language model agents, 2024, arXiv preprint arXiv:2403.15852.

[33] Y. Cai, R. Li, P. Liang, M. Shahin, Z. Li, Designing LLM-based multi-agent systems for software engineering tasks: Quality attributes, design patterns and rationale, 2025, arXiv preprint arXiv:2511.08475.

[34] Z. Rasheeda, M. Waseema, K.-K. Kemella, M. Saari, P. Abrahamsson, LLM-based multi-agent systems for code generation: A multi-vocal literature review, 2026, arXiv preprint arXiv:2604.16321.

[35] J. He, C. Treude, D. Lo, Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead, ACM Trans. Softw. Eng. Methodol. 34 (5) (2025) 1–30.

[36] X. Sun, D. Ståhl, K. Sandahl, C. Kessler, Quality assurance of LLM-generated code: Addressing non-functional quality characteristics, J. Syst. Softw. (2026) 112885.